

Strategies for Decentralized Resource Management

Michael Stumm

University of Toronto
Toronto, Canada M5S 1A4

Abstract

Decentralized resource management in distributed systems has become more practical with the availability of communication facilities that support multicasting. In this paper we present several example solutions for managing resources in a decentralized fashion, using multicasting facilities. We review the properties of these solutions in terms of scalability, fault tolerance and efficiency. We conclude that decentralized solutions compare favorably to centralized solutions with respect to all three criteria.

1 Introduction

Decentralized resource management in distributed systems has become more practical with the availability of communication facilities that support multicasting. In this paper we consider three resource management problems and present decentralized solutions that use multicasting facilities. We review their properties with respect to scalability, fault tolerance and efficiency. The problems we consider are: (1) finding objects in a distributed system, (2) monitoring host resources for scheduling purposes, and (3) allocating hosts for specific functions.

This paper is structured as follows. Section 2 reviews the uses, costs and advantages of multicasting for decentralized applications. The decentralized naming facility of the V distributed system [4] is presented and used as an example. Section 3 presents a decentralized facility for scheduling tasks in a distributed system and describes its implementation for the V system. Section 4 focuses on our current research in progress. It shows that some tasks

that are part of a parallel computation executing as a task group may have special scheduling requirements. For example, we show that some of these tasks cannot share processors with other processes without significantly degrading the performance of the parallel computation. We then present a decentralized method for allocating hosts to accommodate tasks with these special requirements.

2 Background

Many local area networks support *multicast* transmission, where a packet is delivered to a set of zero or more destination hosts based on a single multicast address. In general, multicast is more efficient than *broadcast*, where a packet is transmitted to *all* hosts, since fewer hosts receive and need to service the packet. It is also more efficient than a sequence of unicast transmissions, both in terms of the number of network packets transmitted and the number of packet events at the transmitting site.

Multicasting has an additional advantage over a sequence of unicast transmissions in that multicast addresses can be used as a binding mechanism by associating function with an address rather than location. For example a multicast address can be used to identify the set of hosts that are running file servers. A client can then contact the file servers (with a single packet) without knowing the identity or number of the individual hosts, but only the appropriate multicast address.

The practicality of multicasting in distributed systems and distributed applications has been well recognized. Efforts are currently underway to provide similar functionality in internetworks [3,9,8]. Distributed operating systems are beginning to provide similar functionality at the process level. For example, the V distributed system [7] provides support for *process groups*, where the processes of a group may span multiple physical hosts [6]. A multicast address is associated with each process group and a process may be a member of multiple groups. A message may be sent to a process group by addressing it with the process group identifier. The message is passed to the processes in the group using the network-level multicasting facilities where appropriate.¹ Two important uses

¹The basic communication model provided by the V kernel is that

of group communication in V are to *query* a process group for information or to *update* a process group with (new) information.

The availability of group communication based on multicasting has made it economically feasible and practical to use fully decentralized solutions for distributed problems, including problems related to resource management. For example, obvious uses of group communication are for querying a set of hosts to find the location of a particular object or to query a set of hosts to determine their load in order to make a scheduling decision.

While decentralized solutions are possible by using broadcasts (as demonstrated by the worm program [19]), the cost of doing so would be prohibitive in large systems if used widely and frequently. Similarly, using unicast communication at the application level often complicates applications to a point where centralized solutions become simpler and more economical.

Nevertheless, the cost of group communication can also be high, if not used prudently. Each multicast transmission will cause an interrupt and require service at every host in the multicast set. Also, each query multicast can cause the reply of many packets, resulting in increased network traffic and an increased number of packet events at the host that transmitted the query. These costs can be reduced in a number of ways:

- cache information to reduce the amount of multicasting necessary.
- generate a reply to a query only if the response contains useful information.
- organize processes in (possibly multiple) groups, such that only minimal sets must be addressed.

As an example, consider the decentralized naming facility of the V system designed and implemented by Cheriton and Mann [4]. This facility is used to locate objects maintained by object managers and identified by clients with high-level names. In this scheme, the global (hierarchical) directory is distributed across all object managers such that each manager stores and maintains that portion of the directory corresponding to the objects and services it implements. When a client wishes to locate an object given its (high-level) name, it multicasts a query to a set of object managers. The manager of the named object will respond to the query with an appropriate reply message.

Clients can significantly reduce the number of queries by caching the information they obtain from the responses to their queries. Whenever a manager responds to a query

of *message transactions* that is initiated by a client *Send* operation that transmits a request message to a *server*. The client blocks until a reply message is returned by the server. The V kernel provides for reliability by retransmitting the message when appropriate, signaling an error when communication with the server breaks down.

Sending to a group is similar except that the client is blocked until at least one member of the group has received the message and sent back a reply. Additional replies can be received with *GetReply*. The V kernel also provides for a *real-time Send* that does not block because no replies are allowed in this case. The message is then sent as an unreliable (best efforts) datagram.

it identifies the prefix of the object name that identifies the root of the name tree for which it exclusively is the manager. This prefix-manager pair is then cached by the client. Hence, whenever a client wishes to locate an object, it first looks in its cache for an entry that maps the name to an object manager. If such an entry is found, the operation and name can be sent directly to the manager in question using unicast network access. (Otherwise, a query multicast is necessary.) The cache effectively stores the last known location of objects and is maintained on a per task basis. Measurements indicate that with a suitable management of the cache, a hit ratio of 99.7% can be achieved in practice [4].²

Client cache entries may become invalid, however, for example when a portion of the name tree moves from one manager to another or when a manager crashes. These invalid entries are detected on use, either through an error indication directly from a manager (when it no longer manages the named object) or an error indication from the communication system (when the manager cannot be reached). In this case, the invalid cache entry is deleted and the named objects manager is again sought by multicasting a query.

The overhead of multicasts are further reduced by associating a process group with each node of the naming hierarchy under which multiple object managers maintain a subtree of the name space. For example every object manager is in the group of managers associated with the root of the naming hierarchy. If a manager, M1, manages the name space of the subtree with the root /A/B and manager M2 manages the name space under /A/C then both M1 and M2 will be a member of the process group associated with /A. Assuming no other manager belongs to this group, then if a client wishes to locate a file "/A/C/D" and knows the group identifier associated with /A, but not the identity of the manager M2, it will query the group associated with /A, affecting only those two hosts on which M1 and M2 are running, rather than all object managers.

Finally, network traffic can be reduced by having only those managers reply to a query that manage the object being sought. Using the above example, manager M1, managing the subtree under /A/B, does not reply to a query for /A/C/D. A negative response carries no useful information for the client, since it does not know the members of the group or the number of managers in the group. Moreover, having processes generate replies only when the reply is useful to the client reduces the number of packets that are simultaneously sent back to the client. This helps avoid possible systematic errors at those clients that have trouble receiving multiple packets that arrive back to back.

The naming example illustrates a number of additional features typical of many decentralized solutions based on multicasting. Firstly, in this application, multicasts need not be reliable, as long as a small number of repeated transmissions results in *group coverage* [3]. A

²A number of additional strategies are used to increase the hit rate. For example, each task inherits its name cache from its creator, thus reducing cache misses at task startup time.

multicast is retransmitted if a response is not received within a timeout period equal to the length of the expected response time. A client can safely assume that an object is not reachable if no response is received from the server after several retransmissions.

Secondly, the naming scheme described is fault tolerant in that a node failure will not affect naming. The failure of a client node will have no effect on the naming mechanism for the rest of the system. If a manager fails then only that portion of the name space become unavailable for which the objects are also unavailable. In contrast, with a centralized name server, objects may become inaccessible when the name server fails. Also, a centralized, but replicated server is more expensive to keep if they are to survive individual crashes.

Finally, the naming mechanism described here is scalable to very large systems consisting of thousands of nodes[4], since multicasting is used only infrequently (0.3% of the time) and often directed only to a small subset of all hosts. In a large system, it is probably advisable in any event to use *scoped multicasts*[3], where the transmission of a packet is also controlled by a distance parameter specified by the client. A client searching for an object is typically interested in a replica of the object in proximity and would initially query only nearby managers and, if unsuccessful, gradually increase the scope of its queries.

3 A Decentralized Scheduling Facility

3.1 Introduction

In this section we present the design of a decentralized global scheduling facility. The goal of such a facility is twofold: (1) to aid the user in exploiting remote resources by running programs at remote sites, and (2) to balance the load among the hosts in order to reduce the average response time of tasks. For example, users on workstations based on older technology will observe a response time improved by an order of magnitude if their programs are executed on newer, faster workstations. Similarly, a computationally intensive application will complete significantly faster on an idle host than on a busy one.

The scheduling facility described here was designed to be scalable to a large number of hosts and to be capable of scheduling at a high frequency so that every task that is started could be scheduled. But the design is based on a number of assumptions. First we assume that facilities exist for remote execution and (possibly) migration of tasks. Further, we assume that tasks execute in a network-transparent execution environment, that is, any instance of a program can safely execute on any one of a number of compatible hosts, behaving identically as far as the user is concerned. Many of today's systems provide such functionality [1,10,14,17,18,20].

We also assume that the cost of file access is equal across all hosts. This is generally the case when files are down-loaded from a network file server. (If this is not the

case then scheduling should be restricted to tasks with a sufficiently long expected execution time, so that the cost of additional file transfers is negligible relative to the total execution time.) Finally, the original design was based on the assumption that the distributed system is mainly workstation based. This is important with respect to the policies we implemented, since workstation based systems have a large amount of computational power that goes mostly unused. For example, our implementation site had over 70 workstations with an aggregate processing power of more than 150 MIPS, yet the average processor load was only about 20% during the busiest times. Only a fraction of the total number of hosts are used at any given time. We expect this low level of average processor utilization to become more pronounced as faster processors (10-80 MIPS) and multiprocessor workstations come to the market in the next few of years.

3.2 Selecting an Execution Site for a Task

The scheduling facility is based on hosts publishing their state information: Every host constantly monitors its own state and multicasts it to all interested parties whenever it changes significantly. The state information of a host includes attributes that define the configuration (i.e. processor type, devices, coprocessors, etc.) and the expected load of its resources, such as processor and memory. Each interested host maintains a cache containing the state information of other hosts. This cache is updated every time new information is received and is consulted every time a task is scheduled. If the system contains very many hosts, then the information of only the N "best" hosts needs to be cached.

A program is scheduled by first determining what executable files are available for that program. There may be several, each for a different configuration or host type. Also, any special requirements an executable file may have, such as minimal memory requirements, is determined. A set is then constructed, consisting of the M most lightly loaded hosts (as indicated by our cache) that are eligible execution sites, as determined by the requirements of the program or specified by the user. The number of hosts considered, M , will be a function of the load distribution, so as to consider only "lightly loaded" hosts. The load measure maintained for each host is appropriately scaled to reflect its processing power.

If the local host belongs to this set, then it is chosen, biasing the choice towards local execution. Otherwise, hosts are randomly chosen from this set and probed to verify that the cached information is still valid (to within a certain degree of accuracy). If a probe identifies an inaccurate cache entry, then the cache is updated and the set is modified appropriately; otherwise that host is chosen for execution. (Our implementation indicates that the cache will be accurate enough so that a second probe will very rarely become necessary. The number of probes can therefore be limited to a small number.) The randomness in the selection process helps avoid *scheduling clashes* where

several hosts simultaneously select the same execution site for their programs.

This scheme uses a policy similar to the *Threshold* policy analyzed by Eager et.al.[11], except that the decision whether to execute a program remotely is made based on approximate global information as opposed to the state of the local system alone. Also, instead of probing hosts completely at random, an attempt to make to guess “intelligently” as to which ones would perform best. Eager et.al. conclude that the Threshold policy which uses only a very small amount of system state information yields performance close to that of a theoretically optimal policy. Our policy had to differ from the Threshold policy to accommodate differences in assumptions concerning the characteristics of the underlying system, especially in terms of workload distribution and heterogeneity.

A key issue is how to measure the load of a host. Two important factors are the load on the processor and the load on the memory system as indicated by the amount of ongoing paging activity (or the amount of free memory on a system without demand-paged virtual memory). Although the processor load is more important with respect to its effects on response time, the effects of the memory load should not be underestimated. Adding a new task can significantly increase the page swapping activity, thereby decreasing the performance of all tasks running on that host. (Another possible load factor, a hosts networking activity, is not considered significant enough, especially since much of it is already captured by the processor load.)

Note that the multicasts used for distributing load information need not be reliable. Stale cache entries are detected and updated on use, i.e. when probing a remote host. Moreover, the facilities function correctly in spite of host failures. Orphaned cache entries (those of crashed hosts) indicating light load are also detected on use: The first host to detect an orphan, multicasts that information to all other hosts. Orphaned entries indicating heavier load are seldomly detected on use. Hence, these entries must periodically be found and disposed of using a separate mechanism for this purpose. (Note that it is not reasonable in larger systems to periodically validate each entry by querying the host in question, as this would cause considerable network traffic if done by each host.³)

3.3 Implementation

We describe the implementation of this scheduling facility for the V distributed system [7]. The V-system consists of a distributed kernel and a distributed collection of server processes. Each host has a *program-manager* that provides program management for programs executing on that host and an *exec-server* that provides a shell-like service for interpreting command lines and starting programs for the user. The program-manager monitors the host on which it is executing and maintains the cache containing state information of all other hosts in the system. It makes

³500 hosts probing each other once an hour would generate 138 packets a second

this information available to all other local tasks.

The load of a host is measured by monitoring the CPU utilization, as measured by time allocated to the idle-process (an eternal process with the absolute lowest runnable priority) and is scaled by a factor representing the processing power of the host⁴. Two exponential smoothing functions with different weights are applied to this value. One tracks longer term trends and is smoother while the other tracks shorter term trends. The available memory is monitored in a similar fashion.

The computed long-term values (together with configuration information) are multicast to all other program-managers, if they differ significantly from the values that were last multicast (i.e. by more than 10% of maximum value). Each host that receives the multicast message updates its cache appropriately, as described in the previous subsection. A newly started host can initialize its cache by obtaining a copy from any one of the other hosts.

To schedule a task, the *exec-server* (or any other program wishing to schedule a task) reads the cache from the program-manager and selects one of the hosts represented therein. A probe is sent to that host by sending a load-request message to its program-manager, asking it to load and start executing the specified program. The basic procedure is the same as outlined in [17]. The load value used to select a host is included in the load-request message. If this value is much lower than the short-term load value, then the request is turned down, and if it is much lower than the long-term value, then besides turning down the request the requester is also asked to correct its cache entry.

Once started, the program is run in its own address space and is provided with a network-transparent execution environment. Except for the time needed for scheduling, the fact that a task may be executing remotely, is transparent to the user⁵. The difference between loading a task locally or remotely is equal to the difference between a local and remote message transaction, since all files are loaded over the network (ca. 1.5 ms vs. 3 ms [7]). Hence, even small tasks can safely be scheduled remotely. Also, a user will not notice any sluggishness when interacting with remotely executing interactive programs, since all inter-process communication is based on message passing that differs only by a factor of two for the local and remote cases (again 1.5 ms vs. 3 ms). In fact, we were surprised to find users on slower workstations notice a significantly improved responsiveness in running interactive tasks, such as editors, remotely on newer, more powerful workstations.

Most file systems, including those used in V, do not have facilities for maintaining information about files, such as execution host type, memory requirements, expected run time, task type, etc. Currently, this information is encoded in the file name (i.e. `/usr/bin/cat.m68k`

⁴CPU utilization is reasonable only because we are dealing with heavily underutilized hosts, for otherwise the utilization would be constant close to 100%. For systems with a higher average load, ready-process queue length averages probably are a better measure[12].

⁵Some programs have location dependent semantics.

or `/usr/bin/cat/vax`) or maintained in separate tables. This is clearly a deficiency. We are currently evaluating a number of proposed solutions for maintaining information about files.

Additionally, searching through directory structures along a given search path on a file server can lead to a large number of messages. For example, in our first implementation almost 100 messages were generated per executable file lookup. This can be alleviated, however, either by introducing a hashed bit-map indicating which executable files exist in a user's search path, or by caching directory entries locally.

Measurement of our implementation indicate that for a system with 50 hosts, 10 state-information messages are broadcast to all hosts per minute during busy periods. This is with a natural workload with a load average of about 20%. Our simulations indicate that the number of broadcast message would increase to about 15 with a higher load average of 60%. Each host will service each message broadcast at a cost of approximately 1.5 to 2 milliseconds in processing overhead (depending on host type). In order to compute the local load average, the program-manager must query the kernel for the time allocated to the idle process. It does so every 5 seconds at a cost of approximately 1 ms.

The scheduling facility described provides the mechanisms for implementing many kinds of policies. Our implementation supported the policy of scheduling every task that is started in the system, (unless the user directs the exec-server to follow a different policy). Initial measurements indicate that the response time of large programs decreased by about 10% on average with this policy. This measure alone is not very meaningful, as it depends heavily on the type of hosts running at the time, the distribution of the arrival rate at the individual hosts, the distribution of the load, the type of tasks that are executed, etc. Because of the low load averages, most tasks started on the more powerful hosts execute locally. The benefit of remotely executing programs is due mostly to offloading tasks from the older, slower Workstations to the newer faster ones. For example, to *TeX* a 20 page document on an otherwise unloaded *Sun-2* workstation requires approximately 240 seconds, but only 100 seconds on an otherwise unloaded *Sun-3*, 150 seconds on a *Sun-3* that already has a load average of 50% (as measured by CPU utilization) and 200 seconds on a 75% loaded *Sun-3*.

3.4 Comparison to Other Policies

Our scheduling is based on a Publishing policy. A number of other policies are also suitable for decentralized scheduling. For example, a Querying policy can be used to query the current load of hosts and schedule a new task to the most lightly loaded host that responds. This approach is similar to that used by the naming facility, except that that the load measure indicated in response to a query is much more volatile and therefore less suited for caching. A query must therefore be multicast each time a task is scheduled. Other policies do not make use of load information when choosing an execution site for a task. For

example, a host is selected randomly with the Random policy and a Cyclical policy attempts to assign tasks to hosts in a round robin fashion.

All of these studies have been well studied using both analytical models and simulations [11,22,21]. Simulations we undertook indicated that (given a workload typical of our system) Publishing and Querying performed comparably under light to medium system loads and that the Querying policy performed slightly better under higher loads and for computationally intensive tasks. Moreover, it was found that the Random and Cyclical policies performed significantly worse.

The major reason for choosing a Publishing based scheme over Querying was scalability. In Publishing based schemes, the number of messages is a function of the number of hosts. The overhead can be tuned to an acceptable level. Our implementation indicated that given the smoothing functions used approximately 20 messages per minute were generated per 100 hosts during busy periods a day, when hosts multicast their state information when the load changes by more than 10%. About four times as many messages were generated when state information is multicasts with load changes of 5%. Our simulations, indicated that the number of messages published would not increase significantly under higher system loads.

Although Querying has the advantage that the most current load information can be obtained, it delays the start of a task until responses are received from a query. Also, in Querying based schemes the number of messages is a function of the number of tasks scheduled. Not all hosts have to be queried each time a task is scheduled — for example, hosts can join a group depending on their load and clients can first query lightly loaded hosts — but the number of messages generated in large systems can easily limit the number tasks that can be scheduled. Figure 1 compares the number of network messages in Publishing to the number of messages in Querying for different scheduling frequencies, assuming only 9 hosts receive and respond to an average query. (Note, however, that all of the messages due to Publishing are multicasts, in contrast to Querying, where only the query is multicast.)

Finally, with Publishing, all packet events are evenly distributed across all hosts, whereas hosts that issue a query incur most of the packet events themselves. Hence, precisely those hosts that want to distribute some of their load will momentarily have an even higher load (due to packet processing) every time they need to schedule.

Our simulations also indicated that centralized policies do not perform better than Publishing or Querying given a workload typical of our system. In centralized schemes the number of messages transmitted is also a function of the number of tasks scheduled (one additional message per task), although non of them are multicasts. With a high rate of scheduling, a centralized scheduling server will potentially become a bottleneck unless it is run on a dedicated server host. This in turn complicates the fault tolerance aspects of such centralized servers.

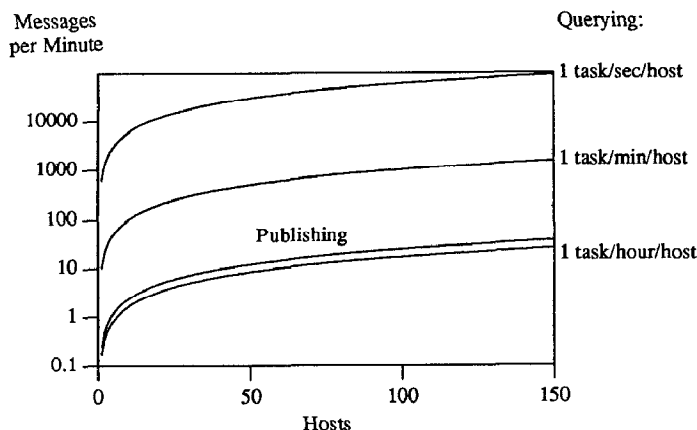


Figure 1: Publishing vs. Querying: number of network packets transmitted.

3.5 Migration

Migration facilities are mainly useful for ejecting all foreign tasks from a host (so that it can be rebooted, for instance). But migration can also be beneficial for load balancing purposes; a task will complete more quickly if it can be migrated to a host with a significant lighter load. This may be necessary, for example, after a scheduling clash, when several hosts simultaneously select the same site for processor-bound tasks.

The scheduling facilities described so far can be extended to handle migration. The global state is periodically reevaluated to decide for each task whether to migrate it to another site. This is associated with more overhead, however, since the local load must be discounted by the proportion due to the process being considered for migration, when comparing the local load to that of a remote host. Hence, the load inflicted by each task must also be measured and monitored.

There are several complications associated with migration. First, there is a danger of instability where tasks migrate from one host to another. This can be solved by requiring the difference in load between the two hosts to be larger than a threshold and by requiring that a task may only migrate if its expected time to completion is large enough. This also has the effect of offsetting the cost of migration by improved performance at the new site.

A second complication is that the measure of the load inflicted by a task must be compatible with the measure of the processor load for the arithmetic to be meaningful. Finally, the decision whether to migrate and to where to migrate cannot always be made by the "system" without additional information from the application. In some instances it is easier to let the application make the migration decisions. For example, some tasks may require efficient access to certain devices and should not be migrated at all or only to a certain subset of the hosts. Others may

have residual dependencies (e.g. a compilation that uses local temporary files) that should also be moved. As another example, the purpose of some tasks are to function as watchdogs, that is, to monitor other tasks to detect their failure (and commence recovery procedures). Since tasks often fail because the host on which they are executing fails, a watchdog should never be migrated to the same host on which the task it is monitoring is executing and vice versa.

4 Allocating Resources for Parallel Programs

In this section we consider some scheduling requirements of parallel programs. Programs can be structured for parallel processing in many ways. Some can be structured to execute on a cluster of hosts connected by a local area network [5,13], treating the cluster as a loosely coupled parallel machine. Workstation clusters are particularly suited for executing parallel computations of this type, since a large portion of the workstations are idle at any given time, especially at night. These parallel computations execute as *task groups*, where the tasks execute in parallel (on different hosts).

Depending on the structure of the computation, the tasks that belong to a task group may require special consideration when being scheduled. We present two such requirements and propose modifications to the scheduling facility of the previous section that accommodate these special requirements.

First we consider programs that execute synchronously, that is, the tasks execute in lock step, synchronizing at the end of each iteration. For example, algorithms for dynamic programming or Gaussian elimination can be structured in this way. Tasks belonging to such synchronous computations, which we call *S-tasks*, should not share the processor assigned to them with any other task. Even slight variations in the processing power made available to an S-task can decrease the performance of the entire computation drastically, because all tasks must periodically wait for the slowest task to complete its iteration and a single, slowly running task can prevent the computation from attaining the expected speedup. For example, consider a parallel synchronous computation (with perfect speedup behavior) executing as 5 tasks, where 4 of them have exclusive use of the processor they are executing on, but one of them has only access to 70% of its processor. This computation will complete more quickly if executed using only 4 tasks if they can run alone on the processors to which they are assigned.

To accommodate for S-tasks in scheduling, we reserve a subset of all hosts to execute S-tasks, at most one at a time. We call these hosts *S-hosts*. In a large system with an overabundance of processing power, an attempt is made to always keep a few S-hosts idle, ready for new S-tasks. (In order not to waste these cycles completely, one can still allow short tasks to be executed on S-hosts, when not being used by an S-task.)

Naturally, the number of S-hosts would have to vary dynamically, adapting to the current state of the system. The number of available S-hosts would be a function of the total number of hosts, the relative number of S-hosts in use, the average load on non-S-hosts, etc. The scheduling facility described in the previous section can be slightly modified to include in the state information whether a host is an S-host or not. Then each host can periodically reevaluate the global and local state to determine if it should become or cease being an S-host, depending on the global state as represented by the local cache. In this scheme, the set of all hosts is partitioned into two disjoint subsets, the S-hosts and the non-S-hosts, in a fully decentralized manner, adapting to the changes in the system. A change in the status of the host is considered significant. A host therefore multicasts its state whenever, it changes from one set to another.

We are currently studying how best to implement these adaptive subsets. A number of tradeoffs are involved. On the one hand, one would like to have the subsets adjust to changes in the system relatively quickly. On the other hand, one would like to minimize the number of transitions and in all cases achieve stability. These tradeoffs are complicated by the fact that hosts base their decisions on partially incorrect information (due to the unreliability of multicasts). We would like to better understand the effects of different choices in threshold values and the effects of the unreliability of multicast transmissions. We are also looking into using a more probabilistic decision procedure to decide whether a host should make a transition from one set to another.

Next we consider scheduling requirements of tasks that are part of a parallel computation with less than perfect speedup behavior. As a general rule of thumb, two tasks that are members of different task groups should not execute on the same processor at the same time. This requirement can be explained by observing that the efficiency of a parallel computation cannot increase with the number of processors; the shape of the speedup curve will always be concave. As an example, consider a parallel alpha-beta searching algorithm within a game of checkers [5]. Figure 2 depicts the average speedup obtained on a cluster of Sun-3's running the V kernel [7]. The speedup with 5 processors (ca. 4) will always be greater than or equal to half the speedup with 10 processors (ca. 3). Therefore, if, for example, we wish to run two such computations at the same time, then it is more efficient (the computations will complete more quickly on average) if each computation used 5 separate processors, rather than if each computation time-shared the same 10 processors. The two computations will also complete less quickly on average if 10 processors first run the tasks of the first computation followed by those of the second computation.⁶

This requirement makes it difficult to schedule many

⁶Using the same line of reasoning, one can also argue that parallel computations should use the most powerful processors in the system, since a parallel computation will always complete more quickly when using two fast processors than if run on 6 processors one third the power of the fast ones, although this may contradict scheduling policies concerning interactive jobs.

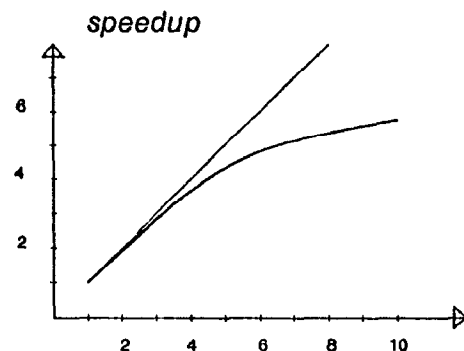


Figure 2: Speedup of Alpha-Beta searching on a workstation cluster.

competing task groups, because either (1) a program is restricted to using a small number of hosts in anticipation that other programs will also run, but limiting the program from exploiting the available processing power, or (2) a program is allowed to "grab" as many hosts as it wishes, potentially blocking out other parallel programs. A third alternative is to require the program to be capable of adjusting the number of hosts it is using dynamically during execution. This alternative is fair, yet allows programs to exploit as much parallelism as is available, but at the cost of potentially having to periodically restructure the workload within the parallel application.

Having self-adjustable parallel programs may not be as difficult as it may sound. Cheriton and Stumm [5] describe a model for structuring parallel computations for a workstation environment that is capable of running while the number of processors available to the computation varies. Some parallel programs, such as simulations described by Jefferson [15], must be capable of dynamically adapting the load on each processor to changing processing requirements. The methodology he uses can be generalized for releasing and adding processors to a parallel computation.

If we wish to implement a policy where each host may execute at most one task belonging to a task group (but may run other "normal" tasks) and where the parallel computations are capable of adjusting the number of tasks they run in parallel, then we propose the following solution. First, tasks belonging to parallel computations register themselves as such with the program-manager on the host on which they are executing. The fact that a host is executing one of these tasks becomes part of the state it maintains and publishes.⁷ (It is in the interest

⁷With tasks executing on hosts to use up the processors "slop", it becomes necessary to modify the definition of the load of a host

of the tasks to register themselves.) A parallel program can determine the number of hosts available for executing its tasks from the local cache of state information and decide how many tasks it should run in parallel and schedule them to execute on an appropriate host.

It may happen that an application would like more hosts than are immediately available. For this purpose we assume that each parallel program in execution is being controlled by a master process. Each such master process joins a well known group and is responsible for negotiating with other master modules and for restructuring the computation if required to do so. If an application would like to use more hosts than are currently available, then it sends a query message to the group of parallel program managers, to determine their identity and the number of tasks they are using. He may request some of these computations to release processors, as long as his task group does not become larger than those he is forcing to restructure. A parallel computation must reduce the number of tasks if asked to do so. A computation releases a processor by migrating its task onto a processor that it already possesses before restructuring. The master process of a computation may periodically reevaluate the global state to determine if it should expand the number of processors it is using.

Hence, in this scheme, task groups compete for processors. If each processor attempts to maximize its number of tasks, then each task group will approximately be equal size. It is possible to refine the scheme by having the master modules specify the speedup characteristics during the negotiation process, to allow computations with better speedup to have more processors. It would also be straight forward to modify the scheme to accommodate task group priorities.

5 Conclusions

We presented several example solutions for problems associated with resource management in distributed systems, including (1) naming, (2) global scheduling and (3) allocating hosts for specific functions. Traditionally, these services have been provided for by centralized servers, i.e. name servers and scheduling servers. We argued that decentralized solutions to these problems have become more practical with the availability of multicasting facilities in modern networks. If these multicasting facilities are used prudently, then decentralized solutions benefit from a number of advantages in terms of scalability, performance and reliability, when compared to centralized solutions, as shown in our examples.

While the use of multicasts generally reduces communication costs (when compared to using broadcasts or a sequence of unicasts) in terms of both network traffic and the number of packet events that must be serviced by hosts, the aggregate cost of all multicasts can become very high in a large system if used indiscriminately. We

to include that proportion of the processor utilization consumed by "normal" tasks and that proportion consumed by tasks belonging to parallel computations.

presented a number of strategies to reduce these costs and showed that by far the most effective reduction of multicast use can be achieved by introducing a local caching mechanism to store information. In each of our examples, application level strategies could be applied to detect stale cache entries or to reduce their effects. Hence, the approaches taken here are all examples of applying Chertou's problem-oriented shared memory[2] techniques.

We believe that the strategies chosen are applicable to many other problems in resource management. As another example, consider a distributed shared virtual memory system [16]. If we assume a page ownership protocol for maintaining consistency, then each page in memory will have an associated owner. Ownership of pages will (generally) change over time. Multicast facilities can be used by the client on a page fault to locate the appropriate owner. The number of multicasts can be reduced significantly in practice, if each client caches the last known owner of each page. This solution is essentially identical to the one proposed by Li [16].

Some of the problems we considered appear to fundamentally require decentralized solutions. For example, in dealing with migration and the scheduling of task groups or parallel programs in execution, we found that some of the scheduling and allocation functions had to be performed by the application, as opposed to a "system" module. The information and algorithms needed to decide what actions to take were too complex and application dependent to be abstracted into simple generic models suitable to be handled by the system (and therefore also centralized servers). In order to make these decisions, applications need efficient access to complex state information concerning a large portion of the distributed system. Since this may entail a large amount of information, it will be obtained more efficiently if maintained on a per host basis rather than a centralized server (especially if this information is made available by using local virtual memory mapping techniques rather than copying).

In conclusion, we view the examples described in this paper to be useful facilities for distributed systems. Their properties with respect to scalability, fault tolerance and efficiency also make them suitable for larger internetwork environments as soon as these environments provide for multicasting facilities. In the future, we expect to see solutions, similar to the ones described here, applied to an increasing number of problems.

Acknowledgments

P. Brundrett helped implement and find bugs in the scheduling facility. S. Shi performed the simulations of the scheduling policies. Discussions with members of the Distributed Systems Group at Stanford, S. Curran, O. Krieger, A. Sanwalka, B. Thomson and S. Zhou were also very helpful.

References

- [1] R. Agrawal and A.K. Ezzat. Location Independent Remote Execution in NEST. *IEEE Trans. on Soft. Eng.*, SE-13(8), 1987.
- [2] D.R. Cheriton. Problem-oriented shared memory: A Decentralized Approach to Distributed System Design. In *Proc. 6th Intl. Conf. on Distributed Computer Systems*, May 1986.
- [3] D.R. Cheriton and S.E. Deering. Host Groups: A Multicast Extension to the Internet Protocol. In *Proc. 9th Data Comm. Symp.*, September 1985.
- [4] D.R. Cheriton and T.P. Mann. *A Decentralized Naming Facility*. Technical Report STAN-CS-86-1098, Department of Computer Science, Stanford University, February 1986.
- [5] D.R. Cheriton and M. Stumm. The Multi-Satellite Star: Structuring Parallel Computations for a Workstation Cluster. *Distributed Computing*, to appear.
- [6] D.R. Cheriton and W. Zwaenepoel. Distributed Process Groups in the V Kernel. *ACM Trans. on Computer Systems*, 3(2), 1985.
- [7] D.R. Cheriton and W. Zwaenepoel. The distributed V Kernel and its Performance for Diskless Workstations. In *Proc. 9th ACM Symp on Operating System Principles*, 1983. appeared in Operating System Review 17(5).
- [8] S. Deering. Host Extensions for IP Multicasting. July 1986. RFC 988.
- [9] S.E. Deering and D.R. Cheriton. Host Groups: A Multicast Extension to the Internet Protocol. December 1985. RFC 966.
- [10] F. Douglass. *Process Migration in the Sprite Operating System*. Technical Report UCB/CSD 87/343, Computer Science Division (EECS), University of California, Berkeley, California 94720, 1987.
- [11] D. Eager, E. Lazowska, and J. Zahorjan. Dynamic Load Sharing in Homogeneous Distributed Systems. *IEEE Trans. Soft. Eng.*, SE-12(5):662-675, 1986.
- [12] D. Ferrari and S. Zhou. *An Empirical Investigation of Load Indices for Load Balancing Applications*. Technical Report UCB/CSD 87/353, Computer Science Division (EECS), University of California, Berkeley, California 94720, 1987.
- [13] R. Finkel and U. Manber. DIB — A Distributed Implementations of Backtracking. *ACM Transactions on Programming Languages and Systems*, 9(2):235-256, 1987.
- [14] R. Hagmann. Process Server: Sharing Processing Power in a Workstation Environment. In *Proc. Principles of Distributed Computing*, 1986.
- [15] D.R. Jefferson and H.A. Sowizral. Fast Concurrent Simulation using the Time Warp Mechanism. In *Proc. of the SCS Dist. Sim. Conf.*, 1985.
- [16] K. Li. *Shared Virtual Memory on Loosely Coupled Multiprocessors*. PhD thesis, Yale University, Dept. of Computer Science, 1986. Tech. Report YALEU/DCS/RR-492.
- [17] M.Theimer, K.A. Lantz, and D.R. Cheriton. Preemptive Remote Execution Facilities for the V-System. In *Proc. 10th ACM Symp. on Operating System Principles*, 1985. appeared in ACM Operating System Review.
- [18] M.L. Powell and B.P. Miller. Process Migration in DEMOS/MP. In *Proc. 9th ACM Symp on Operating System Principles*, 1983. appeared in Operating System Review 17(5).
- [19] J.F. Shoch and J.A. Hupp. The Worm programs - some early experiences with a distributed computation. *CACM*, 25(3), March 1982.
- [20] B. Walker, G. Popek, R. English, C. Kline, and G. Thiel. The LOCUS Distributed Operating System. In *Proc. 9th ACM Symp on Operating System Principles*, 1983. appeared in Operating System Review 17(5).
- [21] Y. Wang and R. Morris. Load Balancing in Distributed Systems. *IEEE Trans. Comp.*, C-34(3), 1985.
- [22] S. Zhou. *A Trace-Driven Simulation of Dynamic Load Balancing*. Technical Report UCB/CSD 87/305, Computer Science Division (EECS), University of California, Berkeley, California 94720, 1986.